DEEP
GUARD

딥러닝 기반
실시간 네트워크
침입 탐지 시스템

DeepGuard

딥러닝 기반 실시간 네트워크 침입 탐지 시스템

Python 3.10.6 | TensorFlow / Keras | Streamlit | CICIDS2017

2-Tier Cascade

MLP → Bi-LSTM

작성자: 홍승현
작성일: 2026. 04. 09 (목)

목차

01 주제 및 선정 이유

02 목표

03 환경 설정

04 데이터셋 구축

05 전처리 과정

06 모델 학습 과정

07 1차 비교 분석

08 1차 모델 선정 + 튜닝

09 2차 비교 분석

10 2차 모델 선정 + 튜닝

11 최종 모델 스펙

12 실제 구현 (Streamlit 시연)

13 트러블슈팅

14 부족한 점

15 향후 계획

16 배운 점

주제 및 선정 이유

DeepGuard — 딥러닝 기반 실시간 네트워크 침입 탐지 시스템

01 선정 배경

사이버 공격은 매년 증가하고 있으며, 알려지지 않은 신종 공격(Zero-Day)에 대한 대응이 핵심 과제로 부상

02 한계

기존 머신러닝(XGBoost, Random Forest)은 학습 데이터에 없는 새로운 공격(Zero-Day Attack)에 취약

↓

시그니처 기반 탐지는 이미 알려진 공격만 막을 수 있음

03 핵심 기술

Autoencoder 기반 비지도학습으로 정상 패턴만 학습

↓

학습 데이터에 없는 미지의 공격도 탐지 가능

04 비지도 학습 선정 이유

실제 환경에서 레이블된 공격 데이터 수집 자체가 현실적으로 불가능

↓

정상 트래픽만으로 학습하는 Autoencoder가 유일한 현실적 해답

05 차별점

단순 분류가 아닌 재구성 오차(MSE) 기반 이상 탐지

↓

공격 유형에 관계없이 정상에서 벗어난 패턴을 탐지하는 더 현실적인 보안 시스템

목표 · 환경 설정 · 데이터셋 구축

목표

- 2-Tier Cascade 실시간 방어 시스템 구축
- 1차: 초고속 필터링 (MLP / CNN / GRU 중 선정)
- 2차: 정밀 심층 검사 (LSTM / Bi-LSTM 중 선정)
- Streamlit 대시보드로 실시간 시연

환경 설정

기술 스택

Python 3.10.6
TensorFlow/Keras · Scikit-learn
Pandas · NumPy · Streamlit

학습 환경

Kaggle Notebook (CPU, RAM 30GB)
※ GPU 미인증 → CPU + 메모리 최적화

개발 환경

VS Code (Cursor) · Local Jupyter
※ MS Store Python → 공식 Python 재설치

활성화 함수

1차(MLP/CNN): relu
→ 초고속/기울기소실방지
2차(LSTM/Bi-LSTM): tanh
→ 긴시퀀스/폭발방지

데이터셋 구축

CICIDS2017
(Canadian Institute for Cybersecurity)

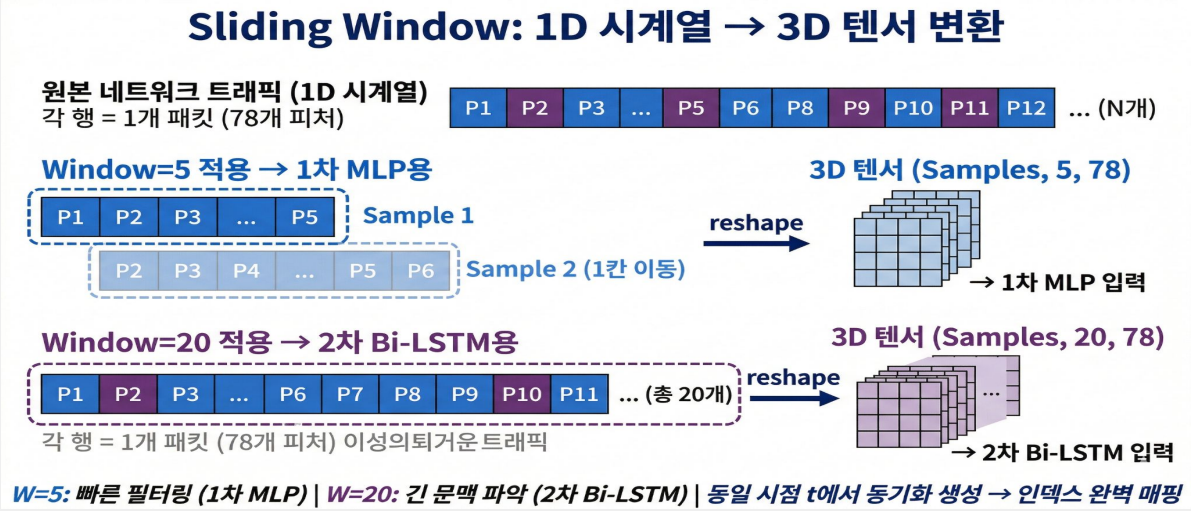
NSL-KDD보다 현실적인 네트워크 트래픽 반영

Monday	BENIGN 100%	학습
Tuesday	Brute Force	테스트
Wednesday	DoS (slowloris)	테스트
Friday-DDoS	DDoS	테스트

전체 79개 피처 → 전처리 후 78개
학습 데이터: 529,481개 중 50% 샘플링 → 약 238,000개

전처리 과정

①	②	③	④	⑤
컬럼명 공백 제거	클렌징 IP, Port 등 식별자 제거 (과적합 방지) 무한대 / 결측치 제거	스케일링 MinMaxScaler - 정상 데이터로만 fit 테스트 데이터는 transform만 적용 (Data Leakage 방지)	Sliding Window (3D 텐서 변환) 1차용: W=5 → (samples, 5, 78) 2차용: W=20 → (samples, 20, 78)	메모리 관리 로컬 RAM 한계로 50% 샘플링 W5/W20 분리 저장 · 테스트 W20: 50,000개



모델 학습 과정

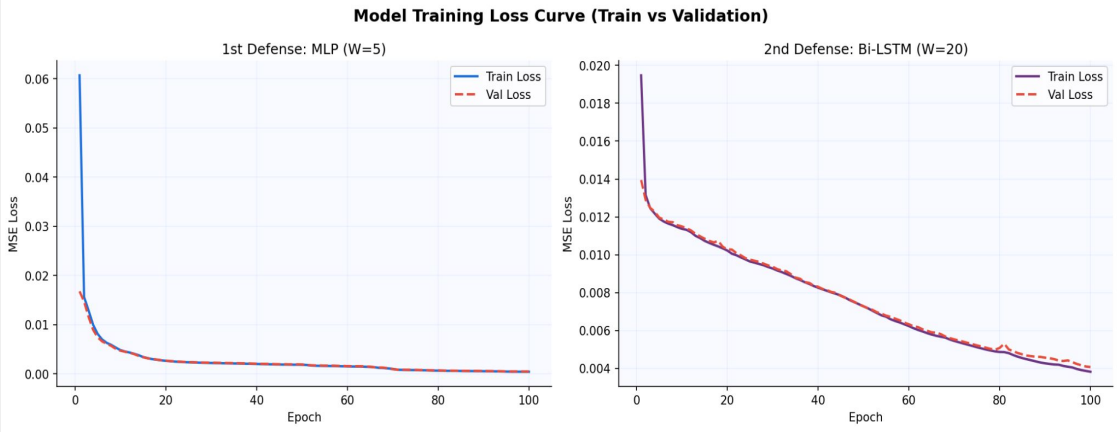
2-Tier 구조

1차 방어선 (W=5, relu)

MLP / 1D-CNN / GRU Autoencoder
→ 속도 우선, 기울기 소실 방지

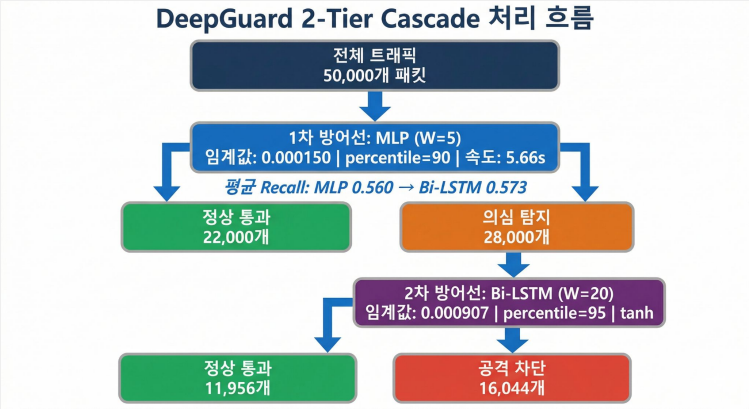
2차 방어선 (W=20, tanh)

LSTM / Bi-LSTM Autoencoder
→ 정밀도 우선, 기울기 폭발 방지



공통 설정

- epochs=100 · batch_size=256
- EarlyStopping (patience=5)
- 임계값: 95th percentile MSE
- 랜덤 시드 고정: np / random / tf (seed=42)



2-Tier 설계 근거 : 1차 MLP 5.66s vs 2차 Bi-LSTM 120.77s (20배 차이)
Bi-LSTM 단독 운용 시 실시간 탐지 불가 → 2-Tier Cascade 필수

1차 비교 분석 (Window=5, relu)

모델	BruteForce	DoS	DDoS	평균Recall	에폭당속도
MLP ★	0.066	0.714	0.835	0.538	5.66s
1D-CNN	0.055	0.712	0.836	0.534	9.32s
GRU	0.059	0.696	0.790	0.515	21.02s

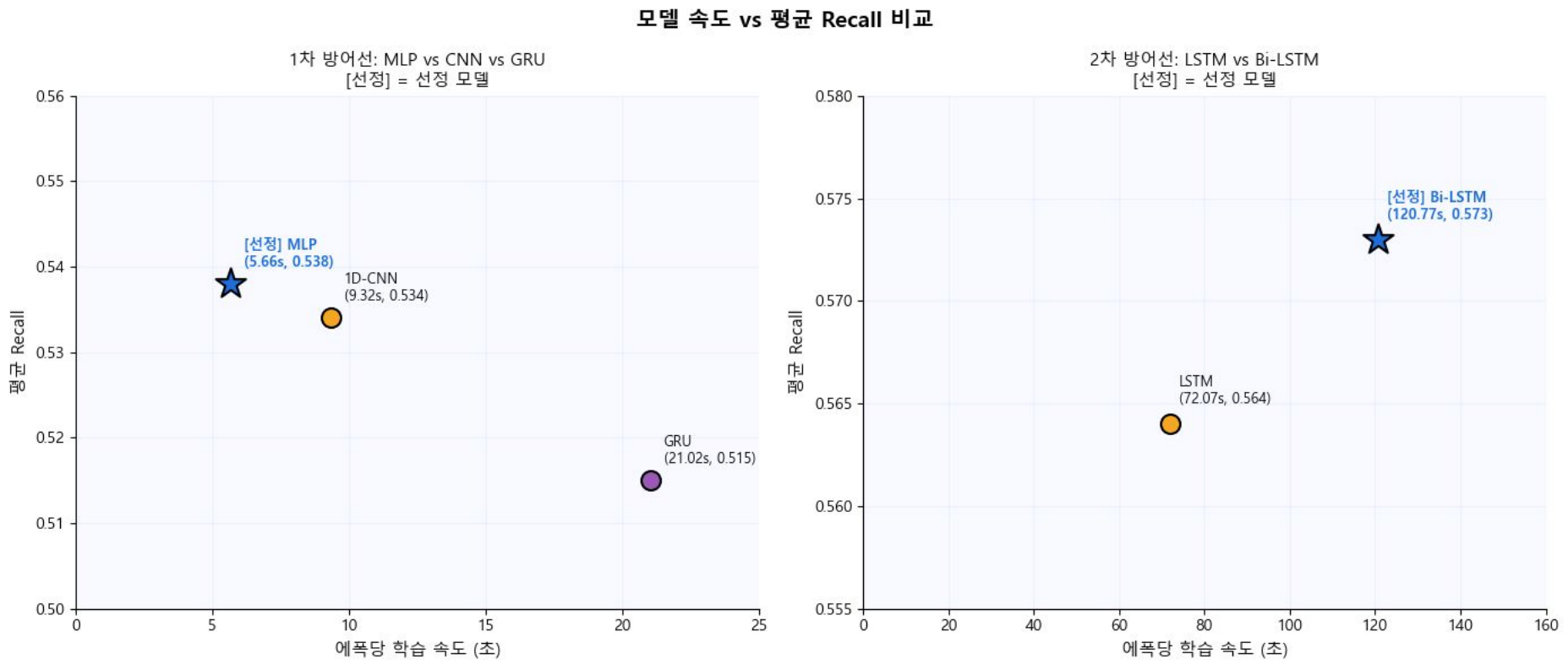
2차 비교 분석 (Window=20, tanh)

모델	BruteForce	DoS	DDoS	평균Recall	에폭당속도	임계값
LSTM (tanh)	0.170	0.637	0.885	0.564	72.07s	0.002361
Bi-LSTM (tanh) ★	0.135	0.681	0.905	0.573	120.77s	0.000907

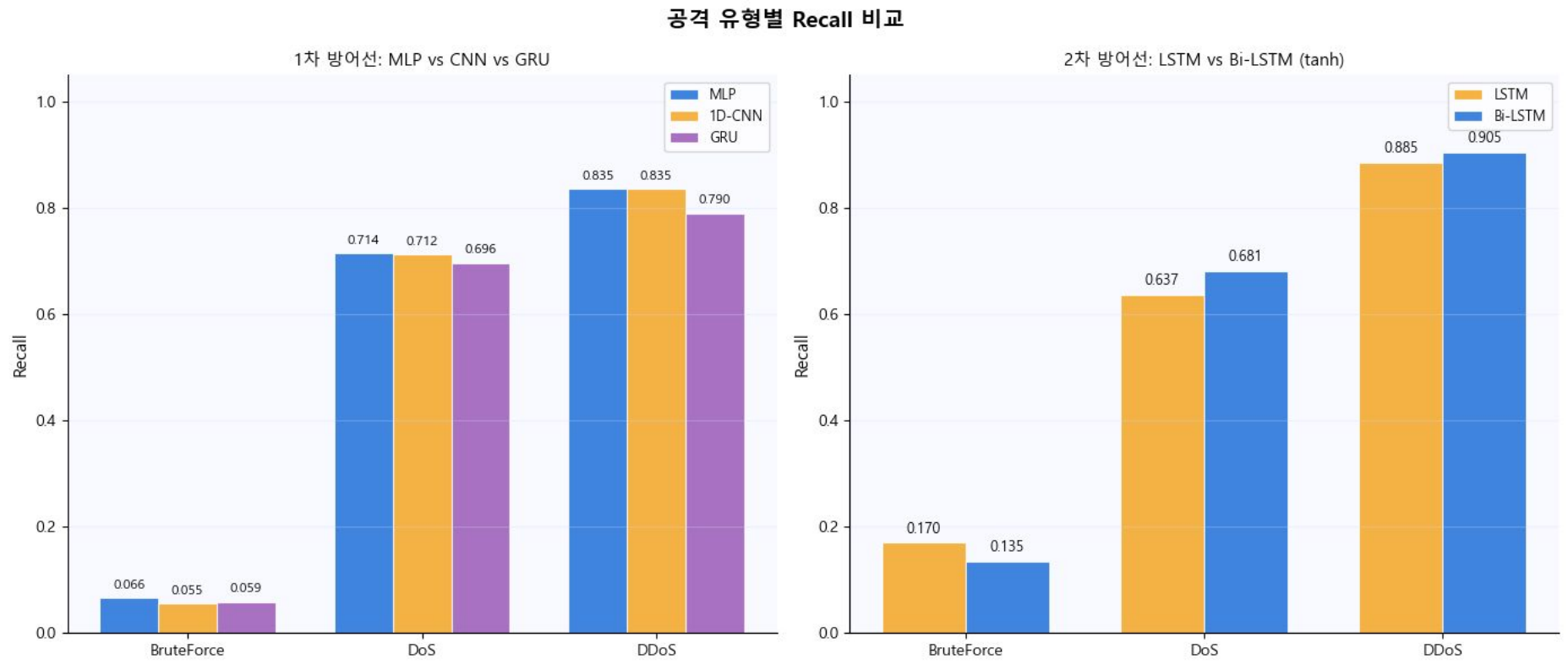
2차 특이사항

- tanh 적용 후 LSTM 학습 효율 38% 향상 (relu 114.86s → tanh 72.07s)
- Bi-LSTM 임계값이 더 낮음 → 정상 데이터를 더 정밀하게 복원

1차 - 2차 비교 분석


모델 속도 vs 평균 Recall 비교
1차 방어선: MLP vs CNN vs GRU
[선정] = 선정 모델
[선정] MLP
(5.66s, 0.538)
1D-CNN
(9.32s, 0.534)
GRU
(21.02s, 0.515)
평균 Recall
에폭당 학습 속도 (초)
0.56
0.55
0.54
0.53
0.52
0.51
0.50
0
5
10
15
20
25
2차 방어선: LSTM vs Bi-LSTM
[선정] = 선정 모델
[선정] Bi-LSTM
(120.77s, 0.573)
LSTM
(72.07s, 0.564)
평균 Recall
에폭당 학습 속도 (초)
0.580
0.575
0.570
0.565
0.560
0.555
0
20
40
60
80
100
120
140
160

1차 - 2차 비교 분석


공격 유형별 Recall 비교
1차 방어선: MLP vs CNN vs GRU
MLP
1D-CNN
GRU
Recall
1.0
0.8
0.6
0.4
0.2
0.0
0.066
0.055
0.059
0.714
0.712
0.696
0.835
0.835
0.790
BruteForce
DoS
DDoS
2차 방어선: LSTM vs Bi-LSTM (tanh)
LSTM
Bi-LSTM
Recall
1.0
0.8
0.6
0.4
0.2
0.0
0.170
0.135
0.637
0.681
0.885
0.905
BruteForce
DoS
DDoS

1차 모델 선정 + 튜닝 (MLP Autoencoder)

선정 근거: MLP ✅

- Recall 차이 최대 0.004 → 통계적으로 유의미하지 않음
- 1차 방어선 목적 = 초고속 필터링
- 속도: MLP(5.66s) vs CNN(9.32s) → 40% 빠름

모델	BF	DoS	DDoS	Recall
BASE (u64, p95)	0.066	0.714	0.835	0.538
TUN1 (u128, p95)	0.064	0.707	0.824	0.532 ↓
TUN2 (u64, p90) ✅	0.128	0.716	0.836	0.560 ↑
TUN3 (u128, p90)	0.117	0.650	0.686	0.484 ↓
TUN4 (u64,p95,drop)	0.069	0.706	0.801	0.525 ↓

핵심: 임계값 완화(p90)가 BruteForce 2배 향상 — units 증가/Dropout 효과 없음

2차 모델 선정 + 튜닝 (Bi-LSTM Autoencoder)

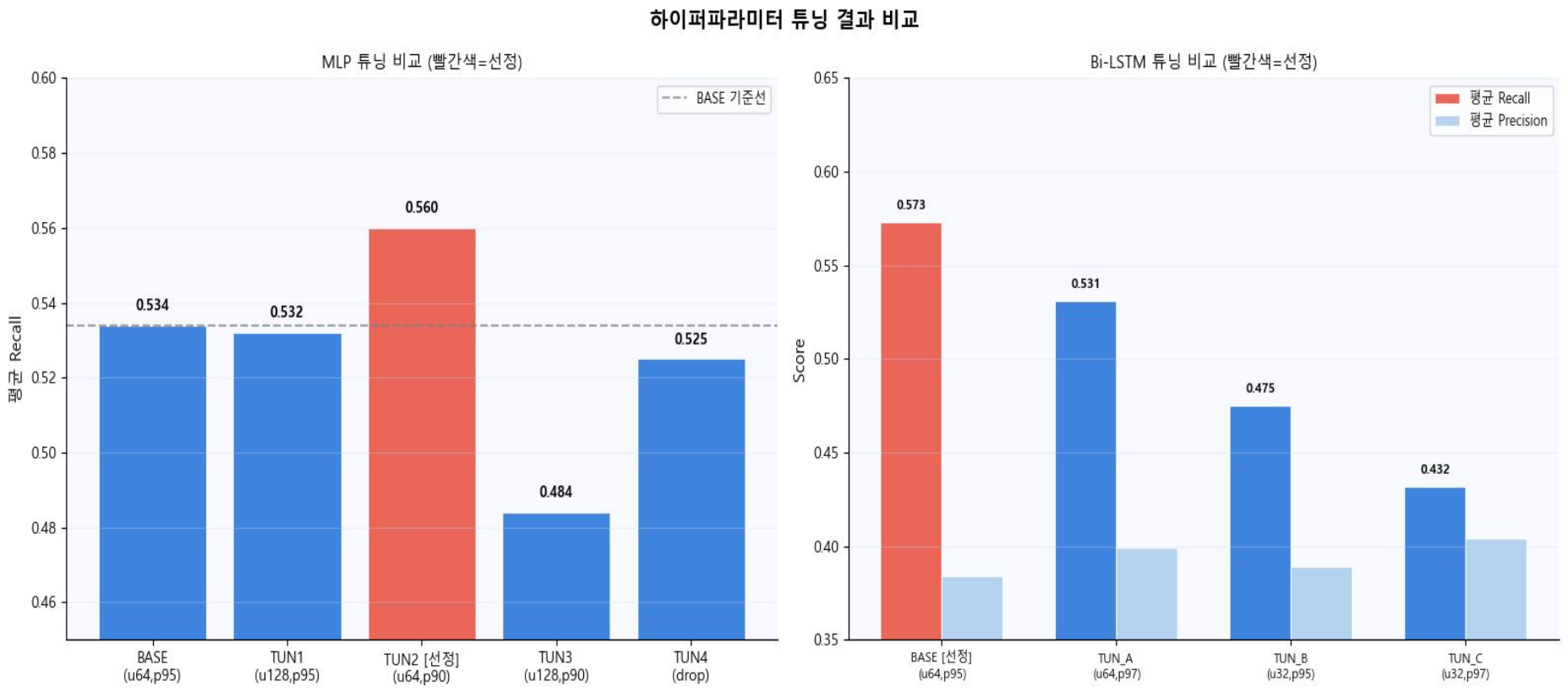
선정 근거: Bi-LSTM

- 평균 Recall 0.564 → 0.573 (+0.009)
- DoS: 0.637 → 0.681 (양방향 학습이 DoS 패턴 포착에 효과적)
- 네트워크 트래픽의 양방향 특성(TCP Request-Response)에 유리

모델	BF	DoS	DDoS	Recall	Prec.
BASE (u64, p95) ✅	0.135	0.681	0.905	0.573	0.384
TUN_A (u64, p97)	0.091	0.598	0.905	0.531 ↓	0.399
TUN_B (u32, p95)	0.142	0.392	0.890	0.475 ↓	0.389
TUN_C (u32, p97)	0.090	0.335	0.872	0.432 ↓	0.404

BASE 최적 — p97 → Precision 소폭↑ but Recall 급락
units=32 압축 → DoS 0.681 → 0.392 폭락

2차 모델 선정 + 튜닝 (Bi-LSTM Autoencoder)



최종 모델 스펙

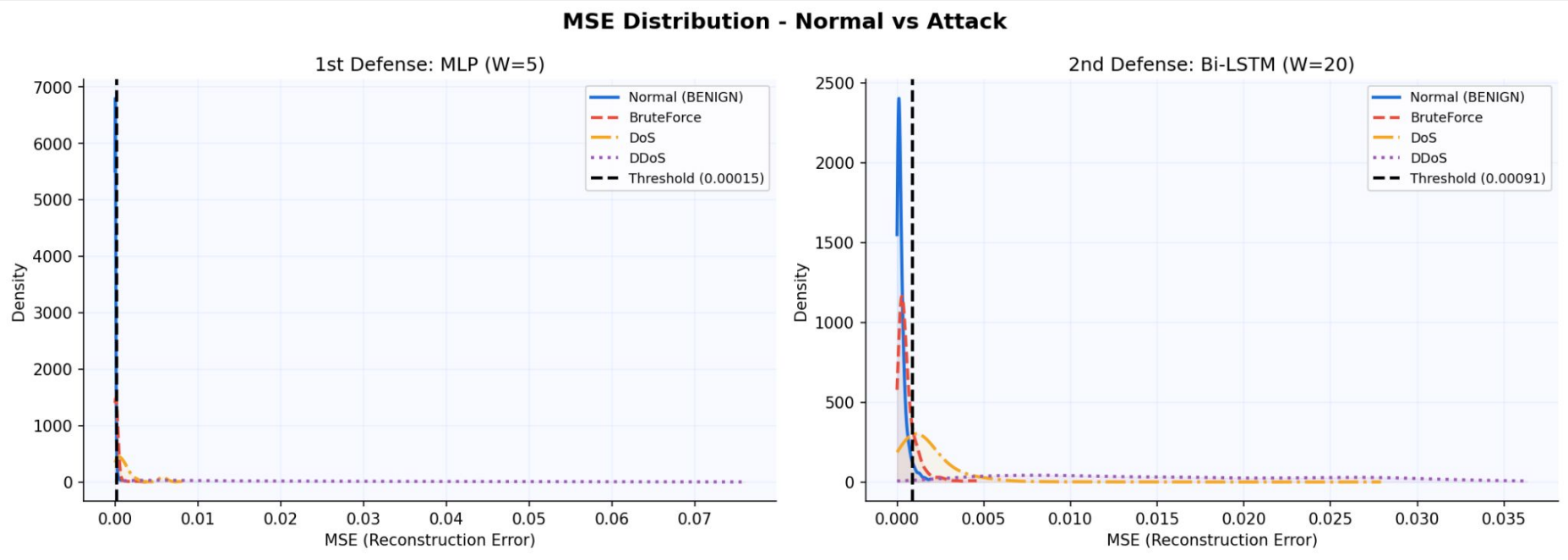
1차 방어선: MLP Autoencoder (TUN2)

- Architecture: Flatten → Dense(128, relu) → Dense(64, relu) → Dense(128, relu) → Reshape
- units=64 · percentile=90 · dropout 없음
- 평균 Recall: 0.560 | 에폭당 속도: 5.66s
- 임계값: 0.000150

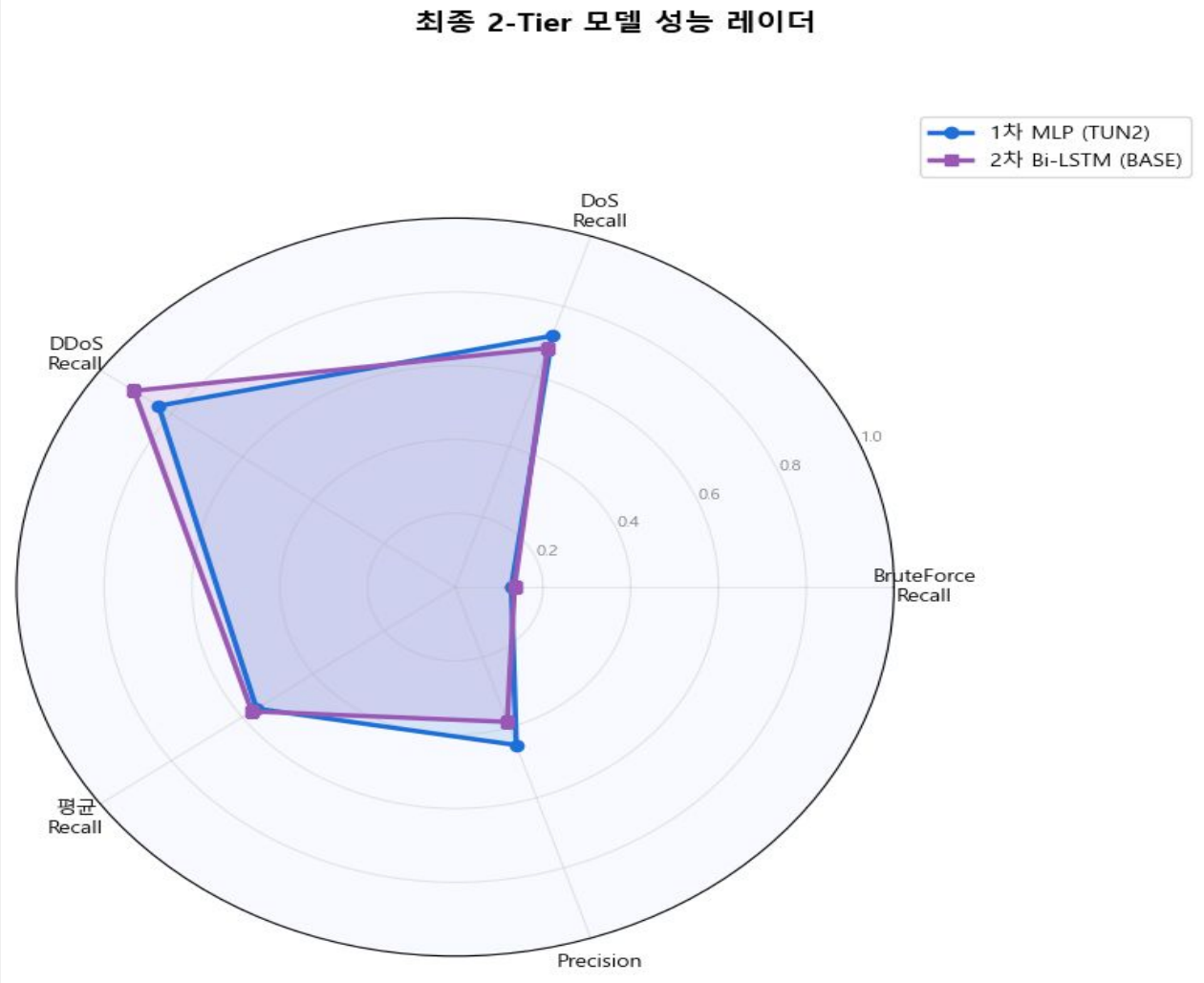
2차 방어선: Bi-LSTM Autoencoder (BASE)

- Architecture: Bi-LSTM(64, tanh) → RepeatVector → Bi-LSTM(64, tanh) → TimeDistributed
- units=64 · percentile=95 · activation=tanh
- 평균 Recall: 0.573 | 에폭당 속도: 120.77s
- 임계값: 0.000907

최종 모델 스펙


MSE Distribution - Normal vs Attack
1st Defense: MLP (W=5)
Normal (BENIGN)
BruteForce
DoS
DDoS
Threshold (0.00015)
Density
MSE (Reconstruction Error)
2nd Defense: Bi-LSTM (W=20)
Normal (BENIGN)
BruteForce
DoS
DDoS
Threshold (0.00091)
Density
MSE (Reconstruction Error)

최종 모델 스펙

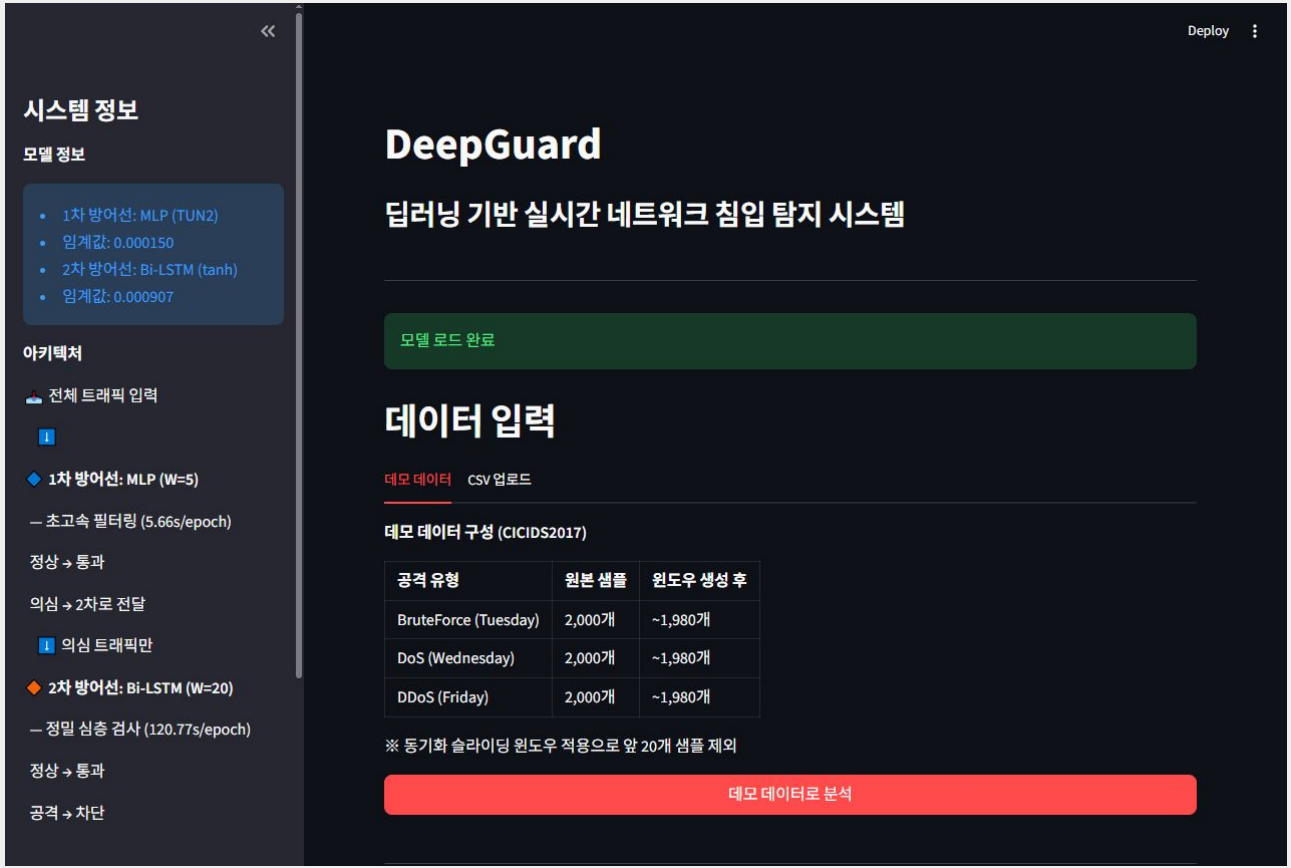

최종 2-Tier 모델 성능 레이더
1차 MLP (TUN2)
2차 Bi-LSTM (BASE)
DoS Recall
BruteForce Recall
Precision
평균 Recall
DDoS Recall
0.2
0.4
0.6
0.8
1.0

실제 구현 (Streamlit 대시보드)

입력	전처리	1차 탐지	2차 탐지	결과
원본 트래픽 demo_raw_3.csv (6,000행 연속추출)	scaler.pkl 자동 스케일링 동일 시점 t에서 W5/W20 동시 생성	MLP Autoencoder 의심 샘플 필터링 (batch_size=256)	Bi-LSTM Autoencoder 정밀 심층 검사	실시간 대시보드 탐지 현황 시각화

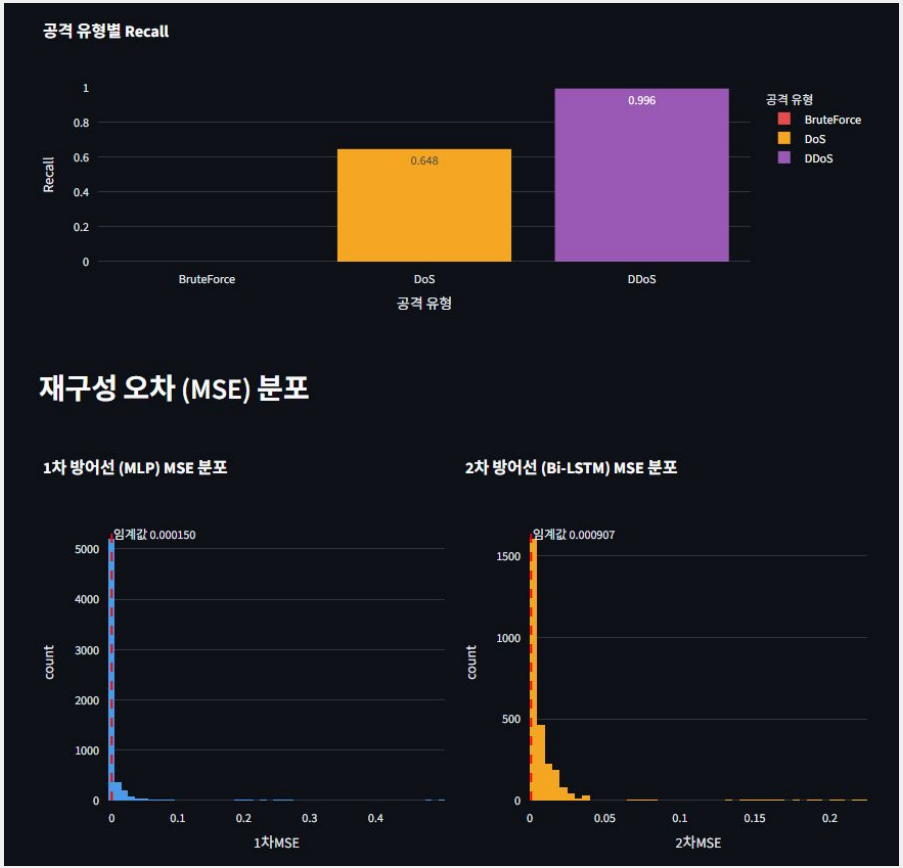
공격 유형	Streamlit 탐지율	Kaggle 학습 결과	비고
BruteForce	0.0%	0.128	비지도 학습 구조적 한계
DoS	64.8%	0.716	Kaggle 68%와 일치
DDoS	99.6%	0.905	Kaggle 90%와 유사

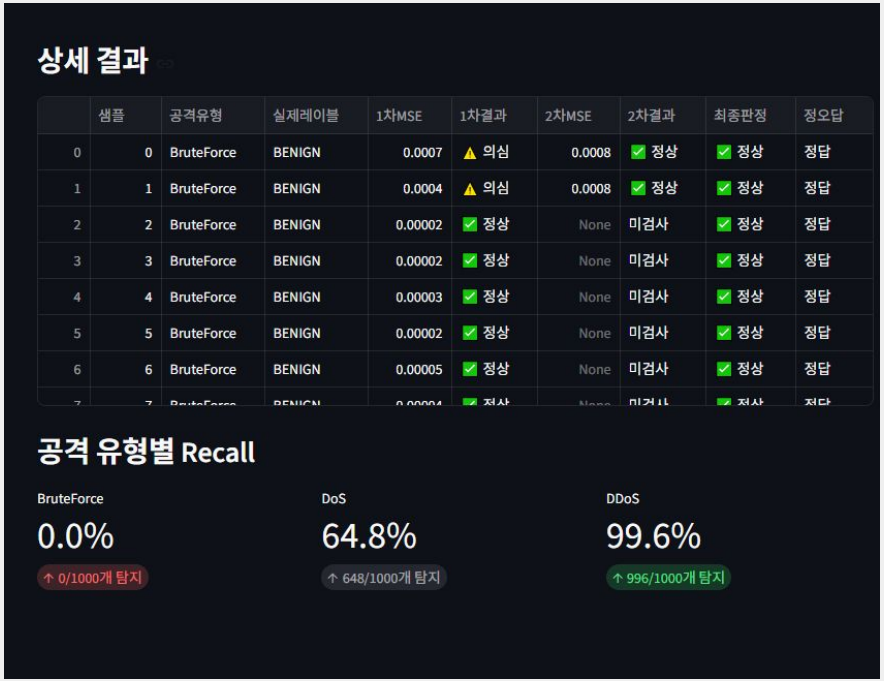
Streamlit 프로토타입 구현 화면 1



Streamlit 프로토타입 구현 화면 2

Streamlit 프로토타입 구현 화면 3





상세 결과

	샘플	공격유형	실제레이블	1차MSE	1차결과	2차MSE	2차결과	최종판정	정오답
0	0	BruteForce	BENIGN	0.0007	⚠ 의심	0.0008	✅ 정상	✅ 정상	정답
1	1	BruteForce	BENIGN	0.0004	⚠ 의심	0.0008	✅ 정상	✅ 정상	정답
2	2	BruteForce	BENIGN	0.00002	✅ 정상	None	미검사	✅ 정상	정답
3	3	BruteForce	BENIGN	0.00002	✅ 정상	None	미검사	✅ 정상	정답
4	4	BruteForce	BENIGN	0.00003	✅ 정상	None	미검사	✅ 정상	정답
5	5	BruteForce	BENIGN	0.00002	✅ 정상	None	미검사	✅ 정상	정답
6	6	BruteForce	BENIGN	0.00005	✅ 정상	None	미검사	✅ 정상	정답

공격 유형별 Recall

BruteForce
0.0%
↑ 0/1000개 탐지

DoS
64.8%
↑ 648/1000개 탐지

DDoS
99.6%
↑ 996/1000개 탐지

트러블슈팅 (총 17개)

01 Colab RAM → Kaggle 전환

02 Kaggle 세션 초기화 → Dataset 저장

03 로컬 W20 전처리 실패 → Kaggle 전처리

04 모델 메모리 누적 → del + gc.collect()

05 MS Store Python → 공식 Python 재설치

06 Kaggle GPU 불가 → CPU + 메모리 최적화

07 1차 랜덤 시드 미설정 → np/random/tf 시드(42)

08 Save & Run All 0초 실패 → 램 재할당

09 파일A / 파일 B 경로 혼동 → os.walk() 탐색

10 결과값 로그 묻힘 → verbose=2

11 ReduceLROnPlateau 성능 저하 → 제거

12 2차 activation=relu 오설정 → tanh 변경

13 relu 기반 비교 신뢰성 부족 → tanh 재학습

14 Streamlit 버튼 클릭 시 초기화 → session_state

15 W5/W20 인덱스 불일치 → 동기화 슬라이딩 윈도우

16 .sample()로 시계열 파괴 → .iloc[] 연속추출

17 1,500개 개별 predict 멈춤 → 배치처리(256)

부족한 점

BruteForce Recall 낮음 (0.13 / 데모 0%)

- 초기 패킷이 정상 로그인과 수학적으로 동일
- 비지도 Autoencoder 구조상 버퍼 누적 전 탐지 불가
- IP/Port 제거 후 순수 통계만으로는 구별 한계

재현성 부족

- 1차 방어선 랜덤 시드 미설정으로 완벽한 재현성 보장 불가
- 동일 코드 재실행 시 미세한 수치 변동 가능성 존재

전체 학습 미진행

- 로컬 RAM 한계로 전체 데이터 100% 학습 미완료
- 50% 샘플링 기반 학습 → 전체 데이터 학습 시 성능 향상 여지 있음

평가 제한

- 테스트 데이터 W20: 50,000개 샘플링 → 전체 평가 미진행
- 단일 데이터셋(CICIDS2017) 의존 → 실제 네트워크 환경 검증 미진행

단일 공격 유형 한계

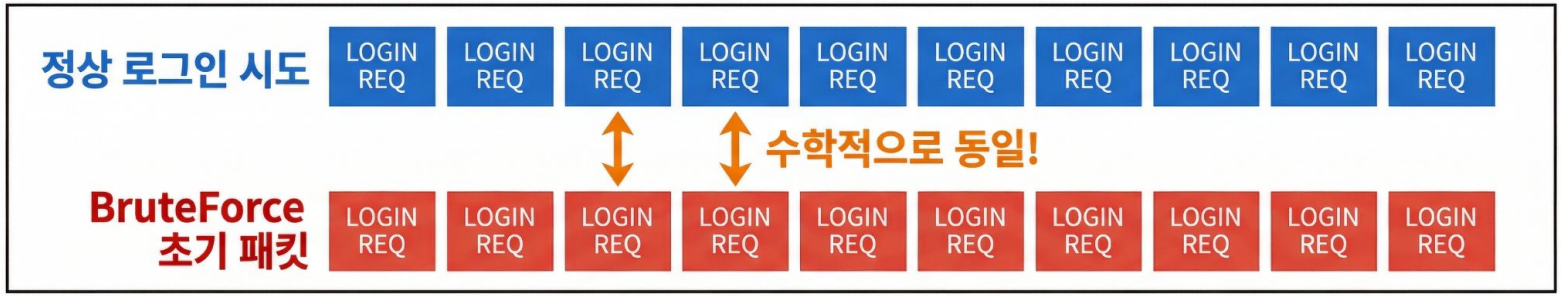
- BruteForce, DoS, DDoS 3가지만 테스트
- PortScan, Botnet, Infiltration 등 다양한 공격 유형 검증 미진행
- 실제 관제 환경에서는 훨씬 다양한 공격 패턴 존재

실시간 패킷 연동 미구현

- 현재는 CSV 파일 기반 시연
- 실제 네트워크 인터페이스에서 실시간 패킷 캡처 → 모델 연동 미완성
- FastAPI 서빙 + 실시간 트래픽 파이프라인 구축 필요

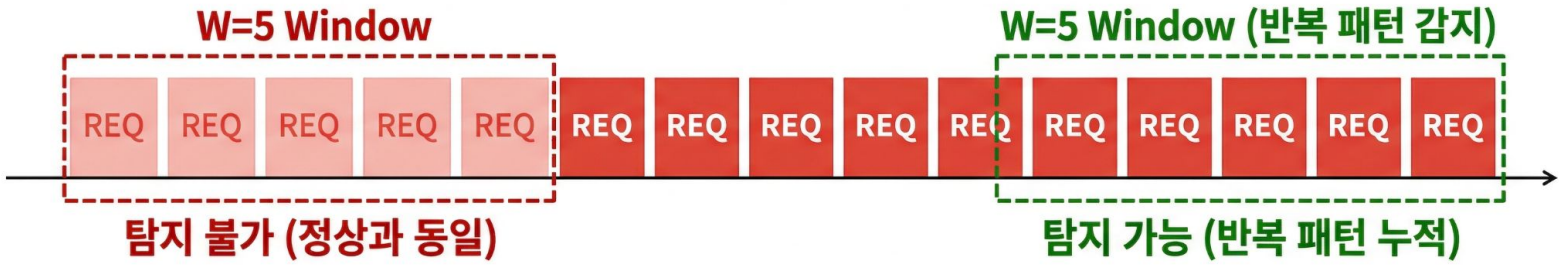
부족한 점

BruteForce 탐지 불가 원인 분석



** IP/Port 제거 후 순수 통계(패킷 길이, 시간 간격)만 남으면 두 패킷은 수학적으로 동일*

시간이 지나 Window에 패턴이 누적되면...



→ 충분한 패킷이 버퍼에 쌓여야 비정상 반복 패턴으로 탐지 가능 (초기 탐지 지연)

향후 계획

BruteForce 탐지율 개선

Window Size 조정 실험

Zero-Day Attack 대응 고도화

미탐지 공격 유형 확대

Java Spring Boot 연동

탐지 결과 DB 저장

실시간 트래픽 연동

FastAPI 서빙

하이브리드 아키텍처 구축

Rule-based (Snort/Suricata) →
BruteForce 즉각 차단
DeepGuard → Zero-Day / DoS / DDoS
지능형 탐지
두 시스템 융합으로 엔터프라이즈급 NIDS 완성

임계값 최적화 자동화

percentile 고정 → F1 기반 동적 임계값

배운 점

비지도학습 이상 탐지

Autoencoder 기반 이상 탐지의 원리와 구조적 한계 체감

대용량 데이터 관리

del + gc.collect(), 분리 처리 전략으로 메모리 절약

2-Tier 설계 기준

모델 선정 기준: Recall vs 속도의 목적별 최적화

클라우드 엔지니어링

Kaggle 환경에서 실무적 데이터 파이프라인 구축

실험 설계의 중요성

1차 교훈을 2차에 맹목적으로 적용하지 않는 독립적 설계

활성화 함수 설계

relu vs tanh — 성능과 학습 효율 모두에 영향

콜백 트레이드오프

ReduceLROnPlateau 역효과 경험 — 콜백 추가 ≠ 항상 성능향상

시계열 데이터 원칙

.sample() vs .iloc[] — 무작위 샘플링은 모델 성능 왜곡

실무 NIDS 이해

패킷 버퍼링 메커니즘 이해 및 동기화 슬라이딩 윈도우 구현

딥러닝의 한계

딥러닝 단독으로는 모든 공격 커버 불가 — Rule-based 융합 필수

THANK YOU

DeepGuard — 딥러닝 기반 실시간 네트워크 침입 탐지 시스템

GitHub: https://github.com/Hong-sh/DeepGuard

작성일: 2026. 04. 09 (목)